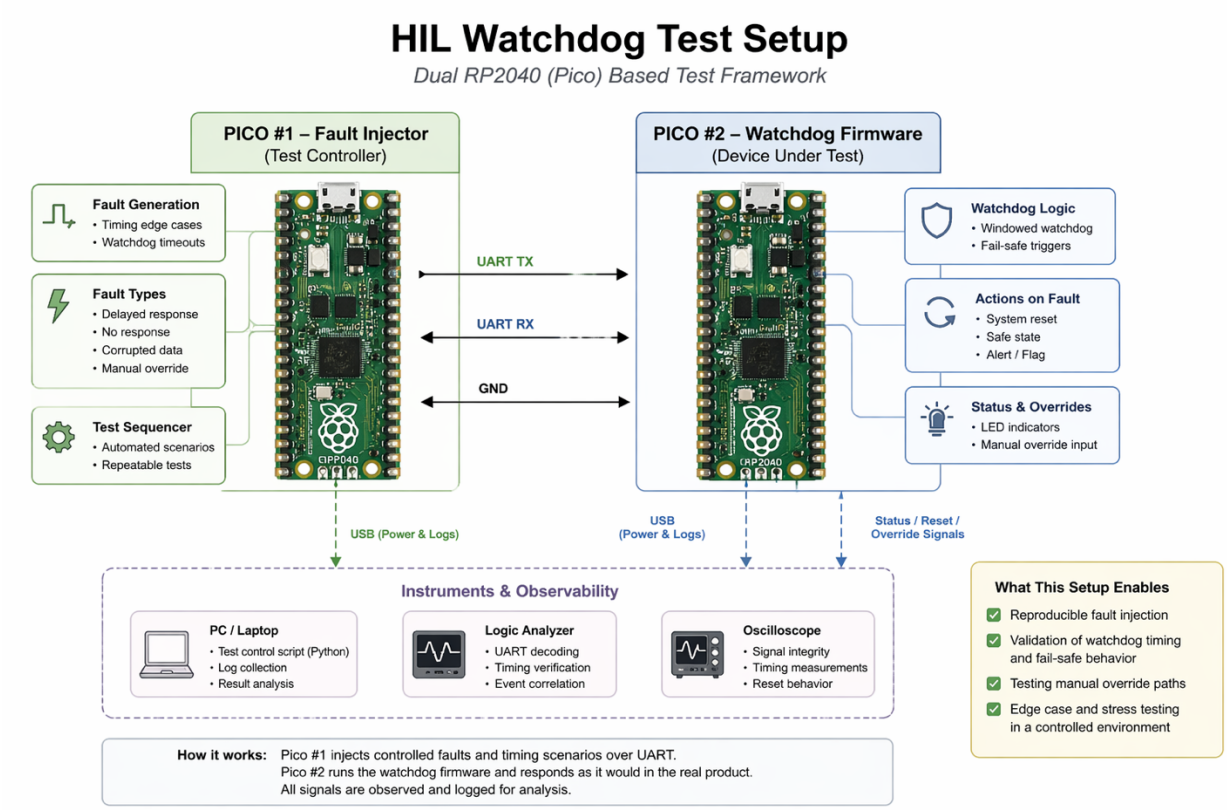
1. One of the projects I'm most proud of is a hardware-in-the-loop (HIL) testing framework I built to validate a watchdog system using two RP2040 microcontrollers. One Pico emulated fault conditions and timing edge cases, while the other ran the actual firmware, allowing me to test fail-safe behavior, reset conditions, and manual override logic under controlled scenarios. I liked this project because it moved beyond unit testing into system-level validation, and it helped catch timing bugs that only showed up under real hardware interaction.



2. My setup is a mix of lightweight tools and whatever helps me debug fastest. I primarily use VS Code for development with C extensions and SSH into embedded Linux systems for on-device work, and I rely heavily on terminal workflows for building, flashing, and logging. For hardware debugging, I use a logic analyzer and oscilloscope alongside UART logs, and I tend to keep simple Python scripts around for quick testing, protocol validation, or automation (for example, Modbus and MQTT testing tools). Over time, I've moved away from IDE-heavy workflows toward more modular setups where I can isolate problems quickly — especially when working across firmware, Linux systems, and hardware simultaneously. I've also started building small internal tools (like HIL test

scripts and protocol debuggers) because they save a lot of time compared to manual testing.

3. First, I try to determine whether the reset is caused by software, power, or external signals. My initial checks are usually simple: monitor the power rails with an oscilloscope for dips or noise, check reset lines for unintended triggers, and add logging over UART to see if the firmware reaches a known failure point before the reset. If the issue is intermittent (like once every few hours), I focus on increasing observability ; adding heartbeat logs, tracking watchdog triggers, and logging timestamps or state transitions leading up to the reset. I'll also try to reproduce it faster by stressing the system (high load, communication bursts, or temperature changes). If I suspect timing or communication issues, I use a logic analyzer to capture SPI/UART/I2C traffic and look for corruption, delays, or protocol violations. For power-related issues, I check regulators, decoupling, and transient behavior during load changes. I change strategy when I stop learning new information for example, if logs don't show anything useful, I shift toward signal-level debugging or isolate subsystems to narrow down the root cause.

4. One thing I think firmware engineers often get wrong is assuming that if the code works once, it's "good enough," especially when dealing with hardware. In reality, a lot of bugs show up only under timing edge cases, power fluctuations, or real-world conditions that aren't covered by basic testing. I also think there's sometimes too much focus on clean abstractions and not enough on observability. In embedded systems, being able to see what's happening through logs, signal probing, or test hooks is often more valuable than having perfectly structured code that's hard to debug. Another thing I've noticed is over-reliance on manual testing. It works early on, but it doesn't scale. Even simple tools like HIL setups or small Python test scripts can make a big difference in catching issues earlier and making debugging reproducible. A piece of common advice I don't fully agree with is "keep firmware as simple as possible." While simplicity is important, I think debuggability and robustness matter more sometimes a slightly more complex system that gives you visibility and control is actually easier to maintain.

5. I've definitely had cases where I added a "temporary" fix that ended up saving a project. In one instance, I was dealing with intermittent communication timing issues over UART that would occasionally cause the system to miss a watchdog window and reset. I didn't have time to fully redesign the timing flow, so I added a guard layer that retried communication and buffered certain operations to avoid hitting the edge case. It wasn't the cleanest solution, but it stabilized the system enough for testing and let us continue development without blocking progress. Later, I revisited the issue and fixed the root cause properly, but I'm still a little proud of how quickly that patch kept things moving.